

SQL Assignment

ClassicModels Retail Database

Submitted by: Aarogya Parajuli

1. Show all customers whose credit limit is greater than 20,000

```
SELECT customerName
FROM customers
WHERE creditLimit > 20000;
```

2. Show employees who report to VP Sales

```
SELECT firstName
FROM employees
WHERE reportsTo = (
    SELECT employeeNumber
    FROM employees
    WHERE jobTitle = 'VP Sales'
);
```

3. Customers in USA with state filled and credit limit between 100,000 and 200,000

```
SELECT customerName, state, creditLimit
FROM customers
WHERE state IS NOT NULL
    AND country = 'USA'
    AND creditLimit BETWEEN 100000 AND 200000;
```

4. Employees who report to Sales Managers

```
SELECT firstName, lastName, reportsTo
FROM employees
WHERE reportsTo IN (
    SELECT employeeNumber
    FROM employees
    WHERE jobTitle LIKE '%Sales Manager%'
);
```

5. Average credit limit of customers per country

```
SELECT country,
    AVG(creditLimit) AS Avg_credit_Limit
FROM customers
GROUP BY country;
```

6. Total orders per date and customer (only where count > 10)

```
SELECT orderDate,
    customerNumber,
    COUNT(orderNumber) AS total_orders
FROM orders
GROUP BY orderDate, customerNumber
HAVING COUNT(orderNumber) > 10;
```

7. Supervisor details and total supervisees (without JOIN)

```
SELECT
    e2.firstName AS supervisor_name,
    e2.jobTitle AS supervisor_job_title,
```

```

        COUNT(e1.employeeNumber) AS total_supervisees
FROM employees e1,
     employees e2
WHERE e1.reportsTo = e2.employeeNumber
GROUP BY e2.employeeNumber;

```

8. Supervisor details and total supervisees (with JOIN)

```

SELECT
    sup.firstName AS supervisor_name,
    sup.jobTitle AS supervisor_job_title,
    COUNT(emp.employeeNumber) AS total_supervisees
FROM employees emp
JOIN employees sup
ON emp.reportsTo = sup.employeeNumber
GROUP BY sup.employeeNumber;

```

9. Customers with credit limit greater than average (using WITH clause)

```

WITH credit AS (
    SELECT AVG(creditLimit) AS avg_credit_limit
    FROM customers
)
SELECT customerName
FROM customers, credit
WHERE customers.creditLimit > credit.avg_credit_limit;

```

10. Rank customers by credit limit and find the 3rd highest

```

WITH credit_ranks AS (
    SELECT customerName,
           creditLimit,
           RANK() OVER (ORDER BY creditLimit DESC) AS rank_no
    FROM customers
)
SELECT customerName, creditLimit
FROM credit_ranks
WHERE rank_no = 3;

```

11. Total employees per office

```

SELECT officeCode,
       COUNT(employeeNumber) AS total_employee
FROM employees
GROUP BY officeCode;

```

12. Total customers handled per office

```

SELECT e.officeCode,
       COUNT(c.customerNumber) AS total_customer_visited
FROM employees e
JOIN customers c
ON e.employeeNumber = c.salesRepEmployeeNumber
GROUP BY e.officeCode;

```

13. Total payments received by each office

```

SELECT o.officeCode,
       o.city AS office_name,
       o.state,
       o.country,

```

```

        SUM(p.amount) AS total_amount
FROM offices o
JOIN employees e ON e.officeCode = o.officeCode
JOIN customers c ON c.salesRepEmployeeNumber = e.employeeNumber
JOIN payments p ON p.customerNumber = c.customerNumber
GROUP BY o.officeCode, o.city, o.state, o.country;

```

14. Total sales by each office

```

SELECT o.city,
       SUM(od.quantityOrdered * od.priceEach) AS total_sales
FROM orderdetails od
JOIN orders o1 ON od.orderNumber = o1.orderNumber
JOIN customers c ON c.customerNumber = o1.customerNumber
JOIN employees e ON e.employeeNumber = c.salesRepEmployeeNumber
JOIN offices o ON o.officeCode = e.officeCode
GROUP BY o.city;

```

15. Total payment pending for each office

```

SELECT ord.orderNumber, ord.customerNumber
FROM orders ord
WHERE ord.customerNumber NOT IN (
    SELECT customerNumber FROM payments
);

```

16. Credit limit proportion per country

```

SELECT customerName,
       country,
       creditLimit,
       creditLimit / SUM(creditLimit) OVER (PARTITION BY country) AS proportion
FROM customers;

```

17. Create a view: customer name, address, and total orders

```

CREATE VIEW customer_order_details AS
WITH customer_orders AS (
    SELECT customerNumber,
           COUNT(orderNumber) AS total_orders
    FROM orders
    GROUP BY customerNumber
)
SELECT c.customerName,
       c.addressLine1,
       c.addressLine2,
       co.total_orders
FROM customers c
JOIN customer_orders co
ON c.customerNumber = co.customerNumber;

```

18. Update customer country

```

UPDATE customers
SET country = 'Nepal'
WHERE customerNumber = 103;

```

19. Delete all payments below 20,000 (with transaction rollback)

```

START TRANSACTION;

```

```
DELETE FROM payments  
WHERE amount < 20000;
```

```
ROLLBACK;
```

20. Insert a new payment for an existing customer

```
INSERT INTO payments  
VALUES (103, 'AB111111', '2026-04-29', 1111);
```

```
SELECT * FROM payments;
```